



A Chunky Size Fits Nobody

Chunking as an Information-Topology Challenge

A part of the series:
Building RAG Before the Model Runs

1/15

Jesper Karlsson · Otrbonita AI Labs

If you read one section, read this one — the short version

Imagine applying the same chunking strategy to both a Mark Twain novel and an Apollo mission radio transcript.

One is dense, continuous narrative prose where a joke's punchline may depend on a setup three paragraphs earlier. The other is fragmented, multi-speaker radio chatter where an entire utterance may consist of the word "Roger." Feed both through the same fixed-size sliding window, and you will damage each of them — but in *different* ways, for *different* structural reasons. That asymmetry is the subject of this paper.

Engineering teams building enterprise RAG pipelines frequently seek a single standardized chunking configuration — a fixed-size sliding window, a static token split — to apply uniformly across heterogeneous vector databases. This paper argues that **no single chunking topology is optimal across materially different information structures**. What can be standardized is the *process* of selecting a chunking architecture; what cannot be standardized is the architecture itself.

The central claim, stated precisely:

Chunking is not fundamentally a tokenization problem. It is an information-topology problem. The retrieval system must understand what kind of information it is indexing before deciding how that information becomes retrievable.

By analyzing two deliberately contrasting document topologies — Project Gutenberg's Mark Twain corpus (dense narrative prose and epistolary collections) and the Apollo mission transcripts (fragmented, multi-speaker time-series dialogue) — we show that information density, semantic dependency range, and structural boundaries call for distinct chunking architectures. Applying a single fixed sliding-window chunker across such heterogeneous sources creates a substantial risk of retrieval degradation, context fragmentation, and loss of the structural boundaries that ground faithful generation.

This paper presents an architectural analysis and a proposed evaluation methodology. It does not yet report measured benchmark results; Section 6 specifies the experiment design in sufficient detail to be executed and falsified.

You can explore these RAG implementations live: **Mark Twain** at mark.otrobonita.com and **The Apollo Program** at spacetalks.otrobonita.com, coordinated through the RAG Mesh at router.otrobonita.com.

1. Scope: Sourcing and Preprocessing Are Assumed Done

Upstream sourcing and preprocessing — the frame on reality, the acquisition strategy, OCR pipelines, format conversion, and clean extraction — are foundational to chunking strategy. This paper deliberately assumes a pristine, structurally tagged input layer *within a coherent frame*. For a detailed analysis of how sourcing chooses that frame and what it means for the system's architecture (specialist vs. mesh node), see *Hunting the Facts*. For a detailed analysis of how preprocessing preserves the frame's topology, see *Human Slop In, AI Slop Out*.

The cascade is:

- **Sourcing establishes the frame** — what reality are we representing? Specialist or mesh node?
- **Preprocessing preserves the topology that frame requires** — timestamps, speaker turns, document boundaries, whatever signals the frame depends on.
- **Chunking respects that topology** — if sourcing framed it as temporal dialogue (Apollo), chunking uses temporal windows; if it framed it as narrative prose (Twain), chunking uses hierarchical narrative units.

To isolate chunking behavior from frame-confusion artifacts, the analysis assumes two prerequisites executed during the fetching and preprocessing phase.

Boilerplate and noise elimination. Source documents undergo automated stripping of non-content structural artifacts. For literary collections such as Project Gutenberg, this entails removing legal headers, license metadata, and transcription footers via boundary matching:

```
import re
# Isolate content between Gutenberg boundary markers
pattern = re.compile(
    r"(?s)\s*\s* START OF TH(?:IS|E) PROJECT GUTENBERG EBOOK.\s*\s*"
    r"(.*)"
    r"\s*\s* END OF TH(?:IS|E) PROJECT GUTENBERG EBOOK"
)
```

For flight transcripts, boilerplate filtering removes telemetry line breaks, page-header timestamps, and tape recorder sync-pulse logs.

Structural scope and boundary preservation. Ingestion pipelines must preserve logical entity boundaries prior to vectorization. In epistolary collections, individual letters must retain their date, recipient, and salutation boundaries to prevent cross-document contamination. In time-series dialogue, Ground Elapsed Time (GET) codes and speaker identifiers (CAPCOM, CDR, CMP, LMP) must be parsed into structured metadata objects attached to each document payload.

Only when source data is properly isolated and structurally tagged can we systematically evaluate how document topology interacts with vector representations and LLM context windows.

2. The Core Concept: Information Topology

The failure of universal chunking stems from a tension between two properties that any chunking strategy must reconcile:

- **Semantic continuity** — the span of text required to express a complete, self-contained unit of meaning.
- **Retrieval granularity** — the token length at which vector similarity matching achieves high precision.

But the deeper principle is not that "different documents need different chunk sizes." It is this:

Different information structures have different relationships between local relevance and global meaning.

In a Twain novel, meaning composes hierarchically along a narrative axis: sentence → paragraph → scene → chapter. In a Twain letter, meaning is bounded by a document container: sentence → paragraph → complete letter. In an Apollo transcript, meaning composes along a temporal and conversational axis: utterance → exchange → temporal episode → mission phase.

These are not merely different chunk sizes. They are **different dependency graphs**. A paragraph from *Roughing It* and a five-minute window of lunar-descent chatter may occupy the same token budget while being entirely different kinds of retrieval unit. This is why chunk size is a **secondary parameter**; chunk **topology** — what the chunk *is*, structurally — is the primary architectural decision.

Two 500-token chunks can be radically different objects:

- **Chunk A**: an arbitrary token slice beginning mid-sentence and ending mid-thought.
- **Chunk B**: a complete conversational episode, bounded by a mission event, with speaker and time metadata attached.

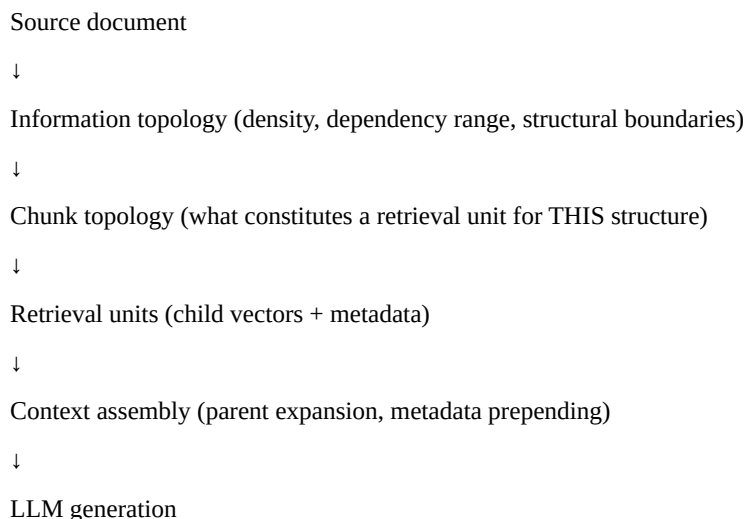
Same token count. Completely different retrieval behavior.

The Topology Alignment Principle

The more closely a retrieval unit reflects the natural dependency structure of its source information, the less structural information must be reconstructed downstream — by the retriever, the reranker, or the generating model itself.

Every recommendation in this paper is an application of this principle, and the information-loss taxonomy in Section 3 enumerates what "structural information" concretely means. The chain the principle implies is: information topology → dominant loss risks → chunk topology → retrieval architecture.

2.1 The pipeline from document to retrieval unit



Every arrow in this diagram is a design decision conditioned on the arrow above it. A universal chunker collapses the entire diagram into a single fixed parameter — and thereby discards the information the pipeline most needs.

3. Chunking as an Information-Loss Function

The most precise way to reason about chunking is to ask: **what information is lost when a document is transformed into retrieval units?** Every chunking strategy is a lossy compression of document structure, and different topologies suffer different loss modes.

Loss mode	What is lost	Example
Boundary loss	A complete thought is severed mid-expression	A satirical setup separated from its punchline
Referential loss	Pronouns and references lose their antecedents	"He," "there," "the previous burn" become unresolvable
Temporal loss	Events become disconnected from their sequence	A caution-and-warning callout detached from the maneuver that triggered it
Speaker loss	An utterance loses its attribution	"Negative" with no record of <i>who</i> said it, to whom
Structural loss	Content loses its container membership	A paragraph no longer knows which letter, chapter, or mission phase it belongs to
Density loss	A chunk carries too much irrelevant material relative to any query	A 1,024-token window in which twelve tokens matter

This taxonomy converts a vague intuition ("chunking matters") into an engineering checklist: for a given corpus, identify which loss modes the information topology is most vulnerable to, then select the chunk topology that minimizes them. The two domains analyzed below are vulnerable to almost perfectly complementary loss modes — which is precisely why they resist a shared strategy.

4. Topological Analysis of Two Contrasting Modalities

The Mark Twain corpus and the Apollo transcripts were chosen because they provide a deliberately strong contrast across several structural dimensions:

Dimension	Mark Twain corpus	Apollo transcripts
Information density	High per sentence	Low per utterance
Discourse mode	Continuous narrative	Fragmented conversation
Dependency range	Long (multi-paragraph)	Short (adjacent exchanges)
Meaning encoding	Implicit, ironic, vernacular	Explicit, technical, procedural
Primary boundary type	Document (letter, chapter)	Temporal (GET, mission phase)
Natural hierarchy	Paragraph → scene → chapter	Utterance → exchange → time block
Dominant loss risks	Boundary, referential, structural	Speaker, temporal, density

4.1 Domain A: The Mark Twain Corpus (Prose, Satire, Epistolary)

Literary prose is characterized by high semantic density, long-range dependencies, indirect thematic references, and vernacular dialect. Satire and irony rely on multi-paragraph setups and delayed payoffs.

Single-turn failure. Isolating single sentences breaks pronoun resolution ("He turned away and wouldn't look at him again") and strips vernacular dialogue of its narrative intent. Primary loss modes: *referential* and *boundary* loss.

Naive fixed-window failure. Fixed-stride token boundaries (e.g., 512 or 1,024 tokens) cut arbitrarily across narrative beats and mid-joke setups. In the epistolary collections, a fixed stride that ignores document containers can merge the closing sign-off of a letter to Olivia Clemens with the opening lines of a letter to W. D. Howells — a *structural* loss that contaminates retrieval with cross-document context.

It is worth being precise about what fails here. The problem is not that sliding windows *inherently* cross document boundaries — a boundary-aware splitter can enforce hard stops at letter edges. The problem is that **naive fixed-size chunking ignores document topology entirely**. The moment an engineer adds boundary awareness, they have made the chunker structure-aware — which concedes the paper's thesis rather than refuting it.

Appropriate topology: hierarchical parent-child chunking. Paragraph-level child vectors (~150–250 tokens) execute high-precision vector retrieval, mapped to parent blocks (~1,000–1,500 tokens for novels; the complete letter for epistolary material) that are passed to the LLM at generation time. The child optimizes similarity scoring; the parent restores semantic continuity. This directly counters boundary, referential, and structural loss.

4.2 Domain B: The Apollo Transcripts (Fragmented Time-Series Dialogue)

Flight transcripts consist of rapid multi-speaker exchanges, short call-and-response radio chatter, dense technical acronyms, and precise time markers:

01 08 20 03 CMP God damn it, there's an outlet down here somewhere.

01 08 20 07 CDR Maybe that'll go behind the - rock box, Al.

01 08 20 10 CMP Ooh!

01 08 20 11 CDR Fit - will that fit on the camera though?

Single-turn failure. Utterances like "Ooh!" or "Roger" carry very low standalone semantic value. Indexed individually, they populate low-information regions of the vector space, degrading retrieval precision — an extreme case of *density* loss, compounded by *speaker* and *temporal* loss when metadata is not attached.

Why aggregation works here. Overlapping dialogue windows (e.g., 512 tokens with 128-token overlap) aggregate rapid exchanges into cohesive conversational units, preserving chronological continuity across speaker transitions. What is a liability in dense prose — the window's indifference to fine-grained meaning — becomes an asset in sparse dialogue, because meaning in this modality *lives in the aggregate*, not the utterance.

Appropriate topology: hierarchical temporal windowing with metadata prepending. These are not competing techniques but layers of one architecture:

Mission phase (e.g., powered descent)

↓

GET parent block (e.g., 5-minute temporal interval)

↓

512-token overlapping dialogue children

↓

Metadata prepended to every child payload

(GET range, speakers present, mission phase label)

The sliding window operates *within* temporal parent boundaries — it aggregates chatter into retrievable episodes, while the GET hierarchy prevents windows from straddling discrete mission phases. So the Apollo recommendation is not "sliding-window *versus* hierarchical." It is a hierarchical architecture whose child-generation mechanism happens to be a sliding window. Which illustrates the larger point:

A chunking strategy is not a single parameter. It is an information architecture.

4.3 The Missing Baseline: Structure-Aware Chunking

A fair critique of any "fixed windows fail" argument is that fixed windows are a weak baseline. Modern production RAG systems increasingly use **structure-aware chunking**: splitting on Markdown headings, paragraphs, speaker turns, timestamps, sections, and scene boundaries.

A terminological distinction is worth drawing here, because the two terms are often conflated. *Structure-aware* means the chunker knows about the source's topology — its document containers, speaker turns, and time codes. *Semantic* chunking, by contrast, uses embedding similarity or model inference to *discover* boundaries where explicit structure is absent. Semantic boundary detection, late chunking, and LLM-assisted segmentation are thus possible *implementations* of structure awareness, not synonyms for it: a GET timestamp is a structural boundary, not a semantic one, and an inferred topic shift is a semantic boundary standing in for missing structure.

These approaches sit within a family of established patterns — parent-document retrievers, hierarchical node parsers, contextual headers — and are typically deployed alongside hybrid dense+sparse retrieval (BGE-M3 natively supports both), rerankers, and long-context generation models that partially absorb imperfect boundaries. This paper's contribution relative to that toolkit is not a new technique but an organizing model: the loss taxonomy and the Topology Alignment Principle explain *why* those patterns work and *when* each is the right choice.

This paper's argument survives — and is sharpened by — including that baseline, because structure-aware chunking is not a counterexample to topology-dependence. It is an *implementation* of it. A splitter that respects headings, speaker turns, and letter boundaries is precisely a chunker that has been made aware of the corpus's information topology. The interesting research question therefore becomes:

Does structure-aware chunking outperform generic fixed-size chunking, and — critically — does the optimal structure differ by domain?

If the answer to the second clause is yes (as the topological analysis predicts), then even the structure-aware family cannot be collapsed into one universal configuration: "split on paragraphs" is the right structural rule for Twain and the wrong one for Apollo, where the operative boundaries are temporal and conversational. The strategies that mitigate imperfect boundaries downstream (rerankers, long context) reduce the *cost* of topology mismatch; they do not eliminate the *advantage* of topology alignment, and they carry their own latency and token-budget costs.

4.4 The Upstream Constraint: Frame and Topology

Before structure-aware chunking is deployed, it is worth stating plainly what it assumes: **the upstream stages have chosen a frame and preserved its topology**. Structure-aware splitting respects document boundaries, speaker turns, timestamps, and section headers — but only because sourcing chose a frame where these signals matter, and preprocessing preserved them.

A chunker that splits on speaker turns assumes sourcing decided the frame is dialogic (Apollo). A chunker that splits on chapter boundaries assumes sourcing decided the frame is narrative (Twain). A chunker that preserves letter boundaries assumes sourcing decided the frame is epistolary.

Different frames demand different structure awareness. This is why chunking is a topology

problem: not all documents are equally structured, not because preprocessing failed, but because sourcing chose different frames and preprocessing preserved them honestly.

If sourcing had chosen a partial, meta-level frame (ragofrags-like), structure-aware chunking might fail entirely — because the sources do not share consistent structure. In that case, the chunker must route through a mesh, not operate as a specialist. The architectural decision was made at sourcing; chunking executes it.

4.5 The Strongest Objection: "Just Detect the Topology Automatically"

The most serious remaining counterargument runs as follows: *a sufficiently sophisticated universal chunker could infer the information topology of any input automatically — therefore chunking can be standardized after all.*

The answer is: yes — and that concedes the thesis rather than refuting it. **What such a system standardizes is the mechanism, not the resulting topology.** A universal ingestion framework with topology detection and routing is entirely achievable, and is in fact the architecture this paper argues for.

The framework is one; the topologies it produces remain many. The claim under attack was never "no pipeline can handle heterogeneous data" — it was "no single chunk topology serves heterogeneous data." A router that classifies inputs and dispatches them to topology-appropriate chunkers is the *strongest possible confirmation* of that claim: its entire reason for existing is that the downstream strategies must differ.

Standardize the intelligence that chooses the topology, not the topology itself.

In practice, the detection layer need not be exotic. A workable classifier can be built from layered signals, escalating in cost only when cheaper signals are ambiguous: rule-based structural features first (timestamp regularity, speaker-label patterns, salutation and sign-off markers, heading density, average sentence length, dialogue-to-prose ratio), then a lightweight document-type model where rules disagree, with LLM-assisted classification reserved for the residue. Most enterprise corpora are far easier to classify than to chunk well — the hard problem was never *detecting* that a flight transcript is a flight transcript; it was deciding what a retrieval unit for one should be. The mixed-corpus condition in Section 6 is designed to test exactly this architecture: whether per-domain routing outperforms any single global strategy.

5. Structural Comparison Matrix

This matrix summarizes the paper's analytical predictions. All rankings are hypotheses pending the Section 6 benchmark; none are measured results.

Corpus / dimension	Single-turn	Naive fixed window	Structure-aware	Hierarchical parent-child
Literary prose (novels)	Poor — referential loss, broken pronouns	Moderate — truncates narrative beats	Good — paragraph/scene boundaries respected	Predicted best — paragraph child → scene/chapter parent
Epistolary (letters)	Poor — fragmented thoughts	Poor — crosses letter boundaries	Good — letter-boundary splitting	Predicted best — paragraph child → full-letter parent
Flight transcripts	Fails — extreme vector noise	Strong — captures rapid turn-taking	Good — speaker/timestamp segmentation	Predicted best — dialogue window child → GET block parent
Vector space impact	Sparse, noisy	Mild boundary slicing	Clean boundaries, variable sizes	High precision, isolated representations
Primary failure mode	Severe context truncation	Arbitrary boundary truncation	Requires reliable structure detection	Indexing and storage overhead

Note the reading of this matrix that matters most: **the structure-aware column requires a different structural rule per row**. Its per-domain success is itself evidence for the thesis.

6. Proposed Evaluation Methodology

This section specifies an evaluation design; the benchmark has not yet been executed. It is presented here so that the paper's claims are falsifiable and so that the experiment can be reproduced independently. Measured results will appear in a subsequent revision.

6.1 Experimental design

The claims in Sections 2–5 predict measurable, domain-dependent differences between chunking architectures. Testing them requires evaluating the *full* retrieval pipeline — not merely whether an LLM can answer correctly when handed correct context.

Corpora. (a) Mark Twain narrative subset (novels), (b) Mark Twain epistolary subset (letters), (c) Apollo transcript subset (e.g., Apollo 11 and 12 descent/surface phases), (d) a deliberately mixed corpus combining all three — the condition under which a universal chunker is most plausibly defensible and therefore the most important test.

Chunking conditions. (A) single-turn/sentence, (B) naive fixed window (512/128 overlap), (C) structure-aware splitting (paragraph/letter boundaries for Twain; speaker-turn/GET segmentation for Apollo), (D) hierarchical parent-child as specified per-domain in Section 4. Retrieval settings (embedding model, k , hybrid weights, prompt template, generation model) are held constant across all conditions so that chunk topology remains the sole independent variable.

Embedding model. BGE-M3 is chosen deliberately for its multi-functionality: it produces dense, sparse, and multi-vector representations from a single model, which allows the benchmark to test whether hybrid retrieval narrows the gap between chunking conditions — i.e., whether lexical matching partially compensates for topology mismatch — without introducing a second model as a confound.

Query set. A minimum of 50 ground-truth question-answer pairs per corpus, authored against source passages, spanning short-range factual lookups and long-range dependency questions (the latter designed to expose boundary and referential loss).

Metrics (RAG-Triad-style; one established evaluation approach among several):

- **Context Precision** — the proportion of retrieved chunks directly relevant to the query. Low precision indicates context-window pollution (density loss).
- **Context Recall** — whether retrieved chunks contain all required ground-truth statements. Low recall suggests that semantic boundaries were severed during ingestion (boundary/structural loss).
- **Faithfulness / Groundedness** — the percentage of generated claims verifiable against retrieved context. Low faithfulness *may indicate* that relevant information was lost, fragmented, or omitted during retrieval — potentially as a consequence of inappropriate chunk boundaries, though poor embedding quality, reranking failure, prompt construction, and model behavior are competing explanations that the ablation design must control for.
- **Answer Relevance** — whether the response addresses the query.

Reporting requirements. Per-condition results tables with corpus size, query counts, retrieval settings (k, hybrid weights), and per-metric scores; statistical comparison across chunking conditions; and cost/latency accounting for the hierarchical conditions, since their indexing and storage overhead is part of the engineering trade-off, not a footnote.

Predicted outcome (pre-registered). Condition D outperforms A–C within each homogeneous corpus on recall and faithfulness; condition B outperforms A on Apollo but underperforms C and D on Twain; no single condition is best across all four corpora — and in the mixed corpus, per-domain routing to topology-appropriate chunkers outperforms any single global condition. If a single condition *does* dominate the mixed corpus, the paper's thesis is weakened and must be revised.

6.2 Evaluation harness (DeepEval)

The following skeleton illustrates the metric harness. In the executed benchmark, `retrieval_context` is populated by the live retrieval pipeline for each chunking condition — never hand-authored, which would test only the generation layer and beg the research question:

```
from deepeval import evaluate
from deepeval.metrics import (
    ContextualPrecisionMetric,
    ContextualRecallMetric,
```

```

FaithfulnessMetric,
AnswerRelevancyMetric,
)
from deepeval.test_case import LLMTestCase

def build_test_case(query: dict, pipeline, chunk_condition: str) -> LLMTestCase:
    """retrieval_context comes from the live pipeline under the
    chunking condition being evaluated - the independent variable."""
    retrieved = pipeline.retrieve(query["input"], condition=chunk_condition)
    answer = pipeline.generate(query["input"], retrieved)
    return LLMTestCase(
        input=query["input"],
        actual_output=answer,
        retrieval_context=[c.text for c in retrieved],
        expected_output=query["expected_output"],
    )
    metrics = [
        ContextualPrecisionMetric(threshold=0.7),
        ContextualRecallMetric(threshold=0.7),
        FaithfulnessMetric(threshold=0.8),
        AnswerRelevancyMetric(threshold=0.8),
    ]
    if __name__ == "__main__":
        for condition in ["single_turn", "fixed_window",
            "structure_aware", "hierarchical"]:
            cases = [build_test_case(q, pipeline, condition) for q in query_set]
            evaluate(test_cases=cases, metrics=metrics)

```

6.3 The full pipeline: frame → topology → chunks

The experiment design in §6.1 tests chunking in isolation (fixed variables, one independent variable). In production, the full dependency chain is:

Sourcing chooses frame → Apollo (temporal, dialogic) / Twain (narrative, epistolary) / ragofrags (partial, meta-level)

Preprocessing preserves topology → GET timestamps and speaker turns / chapter and paragraph structure / whatever coherence exists

Chunking respects topology → temporal windows / hierarchical narratives / meta-level routing

If sourcing chose poorly (frame too loose, boundaries unclear), preprocessing cannot rescue it. If preprocessing corrupted the topology, chunking cannot fix it. Each stage depends on the stage before. The papers are three, but the system is one.

7. Conclusion and Architectural Imperative

Text structure strongly influences the geometry of the resulting vector representations, and no single chunk topology is optimal across materially different information structures. Applying a single fixed chunker across heterogeneous data sources creates a substantial risk of retrieval degradation, context fragmentation, and structural-boundary loss — the upstream conditions under which ungrounded generation becomes more likely.

What *can* be standardized is the pipeline-level discipline: classify the information topology of each corpus, identify its dominant loss modes, and route it to the chunk architecture that minimizes them (§4.5). Standardize the intelligence that chooses the topology, not the topology itself. This is the Topology Alignment Principle in operational form: the routing layer exists precisely because retrieval units must reflect the dependency structure of what they index.

Key recommendations

- **Chunk topology before chunk size.** Decide what a retrieval unit *is* for each information structure before tuning how long it is. Two chunks of identical token count can be entirely different retrieval objects.
- **Match architecture to modality.** Hierarchical parent-child chunking for dense narrative prose and bounded documents (letters); hierarchical temporal windowing with metadata prepending for continuous multi-speaker dialogue.
- **Respect structural boundaries.** Never allow retrieval units to straddle discrete containers — individual letters, chapters, or mission phases. Any chunker that does respect them is, by definition, topology-aware.
- **Decouple retrieval chunks from context chunks.** Optimize small child vectors for similarity scoring; pass large parent blocks to the LLM to preserve generation fidelity.
- **Audit for information loss.** Use the six loss modes — boundary, referential, temporal, speaker, structural, density — as a design checklist when onboarding any new corpus into a RAG pipeline.
- **Treat downstream mitigations as complements, not substitutes.** Rerankers, hybrid retrieval, and long-context models reduce the cost of imperfect boundaries; they do not remove the advantage of topology-aligned chunking, and they add latency and token cost of their own.

The retrieval system should understand what kind of information it is indexing before deciding how that information becomes retrievable. But the understanding begins at sourcing. **Before the chunker can know what kind of information it is indexing, sourcing must have been honest about the frame on reality it chose.** That frame — specialist or mesh node, tight or partial, temporal or narrative — is the foundation on which information topology rests. Chunking's

principle is topology alignment; topology's principle is frame coherence. And frame coherence is the responsibility of the first stage, where the camera angle is chosen.

About This Paper

Disagreement is the point. If part of this is wrong — and some of it will be — the author would rather hear it than not.

Cite as Karlsson, J. (2026). A Chunky Size Fits Nobody. Otrobonita AI Labs.
otrobonita.com/whitepapers.

License Creative Commons Attribution 4.0 (CC BY 4.0). Share it, quote it, argue with it — attribution is all that is asked.

The authoring team

- **Jesper Karlsson** — concept, thesis, ideas, structure, and accountability for every final call.
- **Fable 5, GPT 4.5, Grok 4.5, and Gemini 3.6** — drafting support, fact-checking, handling of uncertainty and counter-evidence, independent critique, and adversarial review.

Seven iterations and approximately thirty-two individual feedback loops.

Written in August 2026.